

Chapter 12 Static and Constant Members

Static keyword: -

Static keyword can be used on: -

1. Local variable of a function (static as a storage class)
2. Member variable of a class
3. Member function of a class

1. Static storage class: -

auto is the default storage class. There is no need to type auto. If a variable has auto storage class, then the life and scope of the variable will be local. If a function is called 3 time, then every time a new local copy of the variable will be created.

static is also a storage class. If a variable has static storage class, then the lifetime of the variable will be equal to the life of the program. It means once a static variable gets memory it will stay in memory till the program terminates. When the function is called first time then the variable will get memory. When the function is called 2nd and 3rd time it will not create new copy of the variable. It will use the previous copy of the variable. There is no need to initialize static variable as it is automatically initialized with 0.

Example of auto storage class: -

```
#include<iostream>
using namespace std;
void f1(){
    auto int a=1;
    a++;
    cout<<a;
}
int main(){
    f1();
    f1();
    f1();
    return (0);
}
```

Output:-

222

Example of static storage class: -

```
#include<iostream>
using namespace std;
void f1(){
    static int a;
    a++;
    cout<<a;
```

Matrix Computers (Page 2 of 11)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
}  
int main(){  
    f1();  
    f1();  
    f1();  
    return (0);  
}
```

Output:-
234

We Provide online/offline training in

C	C++	DSA	JAVA	Adv. Java
Web. Designing	PHP	Python	Django	ML
RSCIT	RSCFA	Office Automation	Tally ERP & Prime	
Advance Excel	DTP	Photoshop	Corel Draw	

Special classes to solve questions of hackerrank.com, hackerearth.com, codechef.com

Free Demo Classes

Repeat Option Available

Free Study Material

We provide internship, summer training for B. Tech students

Type of member variables in a class: -

1. Instance Member Variables: -

- No keyword is required to declare a variable as instance variable.
- There will be as many copies of the instance member variable as there are number of objects.
- The default value of these variables is garbage.
- These variable will get memory as soon as we create object.
- We can access these variables through object only.

- These variables are called instance variables because these variables get memory inside the object which is also called as instance.

2. Static member variables: -

- To declare a variable as static we have to prefix static keyword.
- There will be only one copy of the static member for entire class irrespective of the number of objects and this copy is shared by all the objects of that class.
- The default value is 0.
- Static member variable must be defined outside the class to provide memory. This is necessary because the static data members are stored separately rather than as a part of an object.
- We can access them through object as well as class name and scope resolution operator.
- Since they are associated with the class itself rather than with any class object, they are also known as class variables.
- It is used to store values common for class.

Example: -

```
#include<iostream>
class A{
    public:
        int x;
        static int y;//declaration of static variable no memory
};
int A::y;//defination of static variable memory allocation
int main(void){
    cout<<A::y<<endl;
    A a1,a2,a3;
    a1.x=10;
    a2.x=11;
    a3.x=12;
    a1.y=20;
    a2.y=21;
    a3.y=22;
    cout<<a1.x<<"\t"<<a1.y<<endl;
    cout<<a2.x<<"\t"<<a2.y<<endl;
    cout<<a3.x<<"\t"<<a3.y<<endl;
    return 0;
}
```

OUTPUT:-

```
10    22
11    22
12    22
```

Example: - (Counting objects) In this program when any object is created default constructor is called which increments the "cnt" by 1 and when object is destroyed destructor will decrements "cnt" by 1.

```
#include<iostream>
class counter{
    static int cnt; //      declaration of static variable, no memory
public:
    counter(){
        cnt++;
        cout<<cnt<<"\t";
    }
    ~counter(){
        cnt--;
        cout<<cnt<<"\t";
    }
};
int counter::cnt; //      defination of static variable memory allocation
int main (){
    counter c1,c2,c3;
    cout<<sizeof(c1)<<" byte"<<endl;
    {
        counter c4;
    }
    return 0;
}
```

Type of Member Functions: -

1. Instance Member functions: -

- These member functions are called using object only.
- When we call an instance member function using object then the address of the calling object is automatically passed in the function and this address will be received by "this" pointer.
- Using "this" pointer the function can access all the members of that object.

2. Static member functions: -

- These member functions can be called using class name or using object.
- In static member function no address is passed and "this" pointer will not work.
- Without "this" pointer function can't access instance members.
- A static function can have access to only other static members declared in the same class.

Matrix Computers (Page 5 of 11)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

- Just like the static variables static member functions also does not depend on object of class.
- We are allowed to invoke a static member function using the object and the '.' operator but it is recommended to invoke the static members using the class name and the scope resolution operator.
- Static member functions are allowed to access only the static data members or other static member functions; they can't access the non-static data.
- If we define a static member function outside the class, then we have to prefix static in function declare only not in function definition.

Example: -

```
#include<iostream>
class math{
public:
    static int sum(int,int);
    static float avg(int,int);
};
int math::sum(int x,int y){
    return (x+y);
}
float math::avg(int x,int y){
    return(float(x+y)/2);
}
int main(){
    int x,y;
    cout<<"Enter two numbers ";
    cin>>x>>y;
    cout<<"Sum = "<<math::sum(x,y)<<endl;
    cout<<"Average = "<<math::avg(x,y)<<endl;
    math m1;
    cout<<sizeof(m1)<<" bytes"<<endl;
    int a,b;
    cout<<"enter two numbers ";
    cin>>a>>b;
    cout<<"sum = "<<m1.sum(a,b)<<endl;
    cout<<"average = "<<m1.avg(a,b)<<endl;
    return 0;
}
```

Rules of Local classes: -

1. A local class type name can only be used in the enclosing function

Example: -

```
class A{
```

Matrix Computers (Page 6 of 11)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
public:
    void f1(){
        class B{
            public:
                void f2(){
                    cout<<"local class";
                }
        };
        B b1;//allowed
        b1.f2();
    }
};
int main(){
    A a1;
    a1.f1();
    B b1; //error
}
```

2. All the methods of Local classes must be defined inside the class only

Example: -

```
class A{
    public:
        void f1(){
            class B{
                public:
                    void f2();
            };
            void B::f2(){
                cout<<"local class";
            }
            B b1;//allowed
            b1.f2();
        }
};
int main(){
    A a1;
    a1.f1();
    B b1; //error
}
```

3. A Local class can't contain static data members.

Example: -

Matrix Computers (Page 7 of 11)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
class A{
    public:
        void f1(){
            class B{
            public:
                static int x;
            };
        }
};
```

4. A Local class may contain static functions.

Example:-

```
class A{
    public:
        void f1(){
            class B{
            public:
                static void f2(){
                    cout<<"f2 called";
                }
            };
            B::f2();
        }
};

int main(){
    A a1;
    a1.f1();
}
```

5. Non-static variables of the enclosing function are not accessible inside local classes.

Example:-

```
class A{
    public:
        void f1(){
            int x=5;
            class B{
            public:
                void f2(){
                    cout<<x;//error
                }
            };
        }
};
```

Matrix Computers (Page 8 of 11)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan
9414752750, 9414930104, 9414244351, 0141-2399633

```
        B b1;  
        b1.f2();  
    }  
};  
int main(){  
    A a1;  
    a1.f1();  
}
```

Example:-

```
class A{  
    public:  
    void f1(){  
        static int x=5;  
        class B{  
            public:  
            void f2(){  
                cout<<x;// no error  
            }  
        };  
        B b1;  
        b1.f2();  
    }  
};  
int main(){  
    A a1;  
    a1.f1();  
}
```

6. Local classes can access global types, variables and functions.

const keyword: -

const keyword can be used on: -

1. Any Variable
2. Function Argument
3. Function Return type
4. Member function
5. Object

Note: - Const keyword can be applied on static variables also.

1. Any Variable: -

If we apply const keyword on a variable then we can't change its value.


```
const float PI=3.141f;  
PI=50;           //error
```

2. Function Argument: -

If we add const keyword on an argument of a function then we can't change the value of that argument.

```
void stringcopy(char s1[],const char s2[]){  
    int i=0;  
    while(s2[i]!='\0'){  
        s1[i]=s2[i];  
        i++;  
    }  
}
```

3. Function Return type: -

If we return any constant value then we should add const in the return type.

```
const char * f1(){  
    return "matrix";  
}
```

4. Const Member Functions: -

- If a member function does not alter any data in the class, then we may declare it as a const member function.
- The qualifier const is appended to the function prototype (in both declaration and definition).
- The compiler will generate an error message if such function try to alter the data values.
- Const member function can't change the value of any of its data member but if we want to do so then declare that member as mutable.
- mutable is a new keyword added in ANCI C++.
- When a function is declared as const, it can be called on any type of object, const object as well as non-const objects.

Example : - In this program the display() function is declared as const which means it can't change data members, but you can see that y is declared as mutable so we can change y but if we want to change x the compiler will generate error.

```
#include<iostream>  
class A{  
    private:  
        int x;  
        mutable int y;  
    public:
```

Matrix Computers (Page 10 of 11)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
void getdata();
void display() const;
};
void A::getdata() {
    cout<<"Enter 2 numbers";
    cin>>x>>y;
}
void A::display() const{
    cout<<x<<"\t"<<y<<endl;
    //x=100 error ,non mutable
    y=200; //perfect, mutable
}
int main(){
    A a1;
    a1.getdata();
    a1.display();
    return 0;
}
```

OUTPUT:-

```
Enter two numbers 10    20
10    20
```

Constant Object:-

- If an object is defined as constant then that object can call only constant functions.
- Declaring an object as a "const" means that no one can change the values of its data members.
- whenever a call is made by that object the member function must be of type const otherwise (non const member function can change values of data members) it will generate error.
- Whenever an object is declared as const, it needs to be initialized at the time of declaration. however, the object initialization while declaring is possible only with the help of constructors.

Example: - In this program, getdata() is non const that is why the call to a1.getdata() function will generate error and call to a1.display() will not.

```
#include<iostream>
class A{
    private:
        int x;
        int y;
    public:
        A(){
            x=y=0;
```

Matrix Computers (Page 11 of 11)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
}
A(int x1, int y1){
    x=x1;
    y=y1;
}
void getdata(){
    cout<<"Enter 2 numbers";
    cin>>x>>y;
}
void display() const{
    cout<<x<<"\t"<<y<<endl;
}
};
int main(){
    const A a1(5,6);
    a1.getdata(); //generates error here, call to non const function
    a1.display(); //it is okay, call to const function
    return 0;
}
```